

Elliptic Curve Digital Signature Algorithm (ECDSA)

Vediamo ora nel dettaglio come funziona l'algoritmo di firma digitale per chiavi basate su Elliptic Curve

Sia $(\mathbb{F}_p, \cdot, +)$ il campo dei numeri $\{0, 1, \dots, p-1\}$

Sia $C = \{(x, y) \in \mathbb{F}_p^2 : y^2 = x^3 + ax + b\} \cup \{\infty\}$ una curva ellittica

Sia G un punto sulla curva c.e. l'ordine n del gruppo $\langle G \rangle$ sia primo

Sia (sk, pk) una coppia di chiavi c.e. $sk \in \{1, \dots, p-1\}$ e $pk = sk \cdot G$

Sia $Hash$ una funzione crittografica

Algoritmo di Firma digitale

$Sign(sk, msg)$:

si assume che $h \in n$

$h = Hash(msg)$

generatore sicuro

Sceglie $k \in n$ u.a.r.

$R = k \cdot G$ indicato con le coordinate (r, y_r)

$s = k^{-1}(h + r \cdot sk) \bmod n$

Return $c = (r, s)$

Oss

È cruciale che k rimanga nascosto in quanto se scoperto permette di risalire alla sk

Algoritmo di Verifica

$Verify(msg, pk, c = (r, s))$

$h = Hash(msg)$

$u_1 = s^{-1}h \bmod n$

$u_2 = s^{-1}r \bmod n$

$\tilde{R} = u_1 G + u_2 pk$

Return True se la 1ª coordinata di $\tilde{R}$ è uguale a quella di R

$$s^{-1}hG + s^{-1}rpk = s^{-1}G(h + rsk) = \frac{kG(h + rsk)}{(h + rsk)} = kG = R$$

Segregated Witness

Uno degli aggiornamenti più importanti di Bitcoin è stato il BIP 144 ridenominato Segregated Witness.

Prima di parlare del Dip in se' bisogna definire la differenza fra Softfork e Hardfork

Softfork e Hardfork

Per come è progettata la rete di Bitcoin, questa è riservata ad ogni aggiornamento visto che deve essere adottato da tutti i miner della rete.

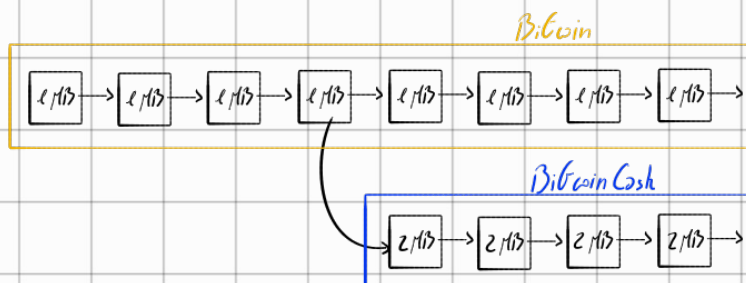
In ogni caso ogni aggiornamento è buona prassi che sia retrocompatibile in modo che non avvenga un fork della catena

Def

Con Hard fork si intende una biforcazione della catena con le rispettive modifiche al protocollo di consenso

Esempio

Un Hard fork è avvenuto quando una parte della comunità di miner decise di raddoppiare la dimensione dei blocchi andando a creare la catena Bitcoin Cash



Def

Con Softfork invece si intendono modifiche più restrittive al protocollo di consenso, che pur non essendo retrocompatibili non causano una biforcazione della catena

Parliamo ora delle modifiche apportate da SegWit alla catena

Motivazioni e modifiche

La motivazione principale che ha voluto risolvere SegWit è quella della Malleabilità delle Transazioni

Questo consiste in una leggera modifica della signature in modo che questa sia ancora valida ma l'hash della Tx no

Per questo è stato deciso di modificare la struttura delle Tx in modo tale che

- gli ScriptSig degli input vengano spostati in una sezione differente della Tx chiamata Witness
 - l'hash della Tx venga calcolato escludendo i Witness
- ↳ Così si è risolto il prob. della malleabilità

Pre SegWit

Versione	# input	input	# output	output	locktime
----------	---------	-------	----------	--------	----------

Post SegWit

↳ Identifica la Tx come SegWit

Versione	Marker	Flag	# output	output	locktime	Witness
----------	--------	------	----------	--------	----------	---------

↳ Definisce la presenza di Witness

Un'ulteriore modifica apportata al protocollo da SegWit è l'unità di misura del blocco.

Questo perché ci si era reso conto che la maggior parte dello spazio dei blocchi veniva occupato dalle signature, per questo l'unità di misura del blocco è cambiata da 1 MB a 4.000.000 wu

↳ Weight Units

Successivamente hanno fatto diversamente i dati:

- Dati base 1 byte $\rightarrow$ 4 wu
- Signature 1 byte $\rightarrow$ 1 wu